

PyTorch 计算图构建



+ 关注他

61 人赞同了该文章

本文来自[How Computational Graphs are Constructed in PyTorch](#)，介绍 PyTorch 中参与创建和执行计算图的部分。

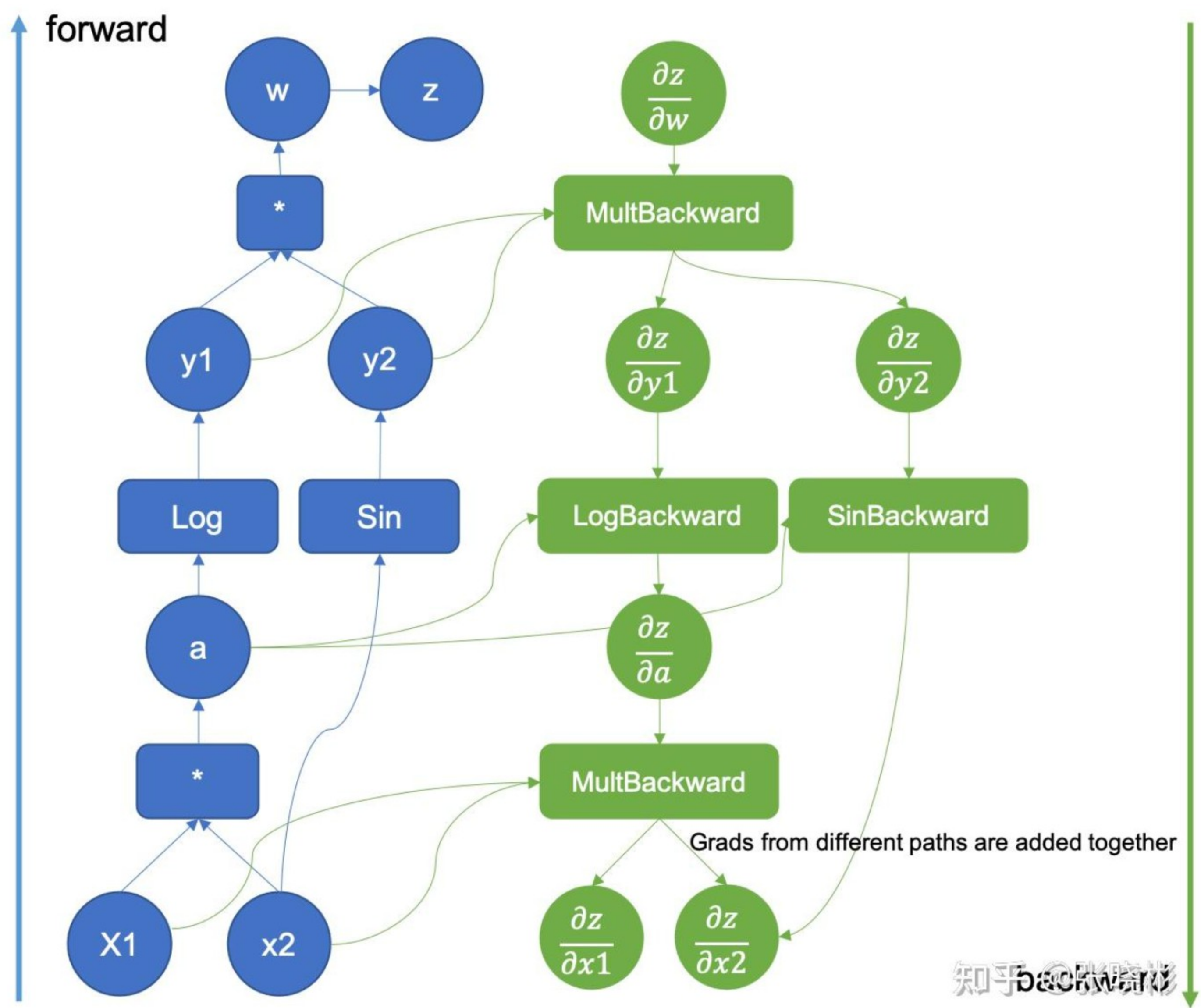
1 Autograd 组件

首先查看 autograd 不同组件的位置，所有路径均相对于 PyTorch 源码的主目录：

- **tools/autograd**：这里可以找到导数的定义，如 [derivatives.yaml](#) 所示，包含若干 Python 脚本，以及一个名为 `templates` 的目录。这些脚本和模板在构建 PyTorch 时，被用来按照 yaml 文件指定的方式，生成导数的 C++ 代码。同事这些脚本也为常规 ATen 函数生成包装器，以便计算图的构建。
- **torch/autograd**：这个目录是直接从 Python 使用 autograd 组件所在的位置。在 [function.py](#) 中，可以找到 `torch.autograd.Function` 的实际定义，这是用户按照文档使用 Python 定制自己微分函数的类。`function.py` 包含为指定函数计算雅可比向量积（Jacobian Vector Product）、Hessian 矩阵、以及其他梯度相关计算的组件。其余文件则包含一些其他组件，如梯度检查器（checker）、异常检测、以及梯度分析器（profiler）等。
- **torch/csrc/autograd**：这是图创建和执行相关的代码存在的地方，这些代码都使用 C++ 编写，因为这是要求极高性能的关键部分。这里有若干文件来实现引擎、元数据存储、以及所有其他需要的组件。此外这里也包含若干以 `python_` 开头的文件，主要功能是允许 Python 对象在 Autograd 中使用。

2 创建图

下面描述 PyTorch 如何用实际代码库来创建计算图。



这一切都始于 Python 代码，当在代码中定义一个需要梯度的张量：

```
>>> x = torch.tensor([0.5, 0.75], requires_grad=True)
```

如果在创建张量时设定 `requires_grad` 标记，C10 会分配一个 `AutogradMeta` 对象，用于保存图信息。

```
void TensorImpl::set_requires_grad(bool requires_grad) {  
    ...  
    if (!autograd_meta_)  
        autograd_meta_ = impl::GetAutogradMetaFactory()->make();  
    autograd_meta_->set_requires_grad(requires_grad, this);  
}
```

`AutogradMeta` 对象在 `torch/csrc/autograd/variable.h` 中的定义为：


```
struct TORCH_API AutogradMeta : public c10::AutogradMetaInterface {
    std::string name_;

    Variable grad_;
    std::shared_ptr<Node> grad_fn_;
    std::weak_ptr<Node> grad_accumulator_;
    // other fields and methods
    ...
};
```

这个结构中最重要字段是 `grad_` 中计算的梯度，以及指向 `grad_fn` 函数的指针，它们会被引擎调用来产生实际的梯度。此外还有一个梯度累加器（accumulator）对象，用来将这个张量在图执行中参与地方的所有不同梯度相加起来。

2.1 图、节点与边

现在如果调用一个以这个张量作为参数的可微函数，相关元数据就会被填充。假定调用一个在 ATen 中实现的常规 torch 函数，令其为两个张量的乘法，结果张量会有一个字段 `grad_fn`，其本质是一个指向计算这个操作的梯度的函数的指针。

```
>>> x = torch.tensor([0.5, 0.75], requires_grad=True)
>>> v = x[0] * x[1]
>>> v
tensor(0.3750, grad_fn=<MulBackward0>)
```

可以看到张量的 `grad_fn` 字段有一个 `MulBackward0` 值，这个函数如 [derivatives.yaml](#) 中定义那样，C++ 代码由 `tools/autograd` 中的脚本自动生成，生成的源码可以在 `torch/csrc/autograd/generated/Functions.cpp` 中找到：

```
variable_list MulBackward0::apply(variable_list&& grads) {
    std::lock_guard<std::mutex> lock(mutex_);
    IndexRangeGenerator gen;
    auto self_ix = gen.range(1);
    auto other_ix = gen.range(1);
    variable_list grad_inputs(gen.size());
    auto& grad = grads[0];
    auto self = self_.unpack();
    auto other = other_.unpack();
    bool any_grad_defined = any_variable_defined(grads);

    if (should_compute_output({ other_ix })) {
        auto grad_result = any_grad_defined
            ? (mul_tensor_backward(grad, self, other_scalar_type))
```

```
        : Tensor();
        copy_range(grad_inputs, other_ix, grad_result);
    }
    if (should_compute_output({ self_ix })) {
        auto grad_result = any_grad_defined
            ? (mul_tensor_backward(grad, other, self_scalar_type))
            : Tensor();
        copy_range(grad_inputs, self_ix, grad_result);
    }

    return grad_inputs;
}
```

grad_fn 对象继承自 `TraceableFunction` 类，是 `Node` 的派生，仅激活一个用于调试和优化的跟踪属性。在定义上图包含节点和边，这些函数实际上是计算图的节点，被 `Edge` 对象连接起来从而实现之后的图遍历。`Node` 的定义可在 `torch/csrc/autograd/function.h` 中找到：

```
struct TORCH_API Node : std::enable_shared_from_this<Node> {
    ...
    /// Evaluates the function on the given inputs and returns the result of the
    /// function call.
    variable_list operator()(variable_list&& inputs) {
        ...
    }

protected:
    /// Performs the `Node`'s actual operation.
    virtual variable_list apply(variable_list&& inputs) = 0;
    ...
    edge_list next_edges_;
```

可以看到它本质上是重载了执行函数调用的 `operator()`，而纯虚函数被称为 `apply`。就像上面 `MulBackward0` 中那样，自动生成函数重载 `apply` 方法。最后节点有一个边列表实现图连接，`Edge` 对象被用来将 `Node` 连接起来，其实现非常直接，只需要一个函数指针，也就是边连接起来的实际 `grad_fn` 对象，以及一个输入数字作为边的 `id`：

```
struct Edge {
    ...
    /// The function this `Edge` points to.
    std::shared_ptr<Node> function;
    /// The identifier of a particular input to the function.
    uint32_t input_nr;
};
```


2.2 将节点连接起来

当调用两个张量的乘积操作时，就进入自动生成代码的领域。tools/autograd 中的脚本会填充一些模板将 ATen 中的微分函数包装起来，这些函数具有在前向传递期间构造反向图的代码。

gen_variable_type.py 脚本负责写所有包装代码，这个脚本在 PyTorch 创建过程中被

tools/autograd/gen_autograd.py 调用，输出自动生成的函数包装器到

torch/csrc/autograd/generated/。现在看一下张量乘法生成的简化代码，从源码编译 PyTorch 时可以在 torch/csrc/autograd/generated/VariableType_4.cpp 文件中找到。

```
at::Tensor mul_Tensor(c10::DispatchKeySet ks, const at::Tensor& self,
                      const at::Tensor& other) {
    ...
    auto _any_requires_grad = compute_requires_grad( self, other );
    std::shared_ptr<MulBackward0> grad_fn;
    if (_any_requires_grad) {
        // Creates the link to the actual grad_fn and
        // links the graph for backward traversal
        grad_fn = std::shared_ptr<MulBackward0>(new MulBackward0(), deleteNode);
        grad_fn->set_next_edges(collect_next_edges( self, other ));
        ...
    }
    ...
    // Does the actual function call to ATen
    auto _tmp = ([&]() {
        at::AutoDispatchBelowADInplaceOrView guard;
        return at::redispatch::mul(ks & c10::after_autograd_keyset, self_, other_);
    })();

    auto result = std::move(_tmp);
    if (grad_fn) {
        // Connects the result to the graph
        set_history(flatten_tensor_args( result ), grad_fn);
    }
    ...

    return result;
}
```

看一下这个代码最重要的几行：首先是通过 grad_fn = std::shared_ptr(new MulBackward0(), deleteNode) 创建 grad_fn 对象；然后是调用 grad_fn->set_next_edges(collect_next_edges(self, other)) 来创建连接节点的边。

```
struct MakeNextFunctionList : IterArgs<MakeNextFunctionList> {
    edge_list next_edges;
    using IterArgs<MakeNextFunctionList>::operator();

    void operator()(const Variable& variable) {
        if (variable.defined()) {
            next_edges.push_back(impl::gradient_edge(variable));
        }
        else {
            next_edges.emplace_back();
        }
    }

    void operator()(const c10::optional<Variable>& variable) {
        if (variable.has_value() && variable->defined()) {
            next_edges.push_back(impl::gradient_edge(*variable));
        }
        else {
            next_edges.emplace_back();
        }
    }
};

template <typename... Variables>
edge_list collect_next_edges(Variables&&... variables) {
    detail::MakeNextFunctionList make;
    make.apply(std::forward<Variables>(variables)...);
    return std::move(make.next_edges);
}
```

给定输入变量（一个常规张量），`collect_next_edges` 函数会调用 `impl::gradient_edge` 来创建 `Edge` 对象：


```
Edge gradient_edge(const Variable& self) {  
    // If grad_fn is null (as is the case for a leaf node), we instead  
    // interpret the gradient function to be a gradient accumulator, which will  
    // accumulate its inputs into the grad property of the variable. These  
    // nodes get suppressed in some situations, see "suppress gradient  
    // accumulation" below. Note that only variables which have `requires_grad  
    // True` can have gradient accumulators.  
    if (const auto& gradient = self.grad_fn()) {  
        return Edge(gradient, self.output_nr());  
    }  
    else {  
        return Edge(grad_accumulator(self), 0);  
    }  
}
```

要理解边的原理，假定先前执行的函数产生了两个输出张量，都设置了 `grad_fn`。每个张量都有一个 `output_nr` 属性，包含它们被返回的顺序。当创建当前 `grad_fn` 的边时，每个输入变量会创建一个 `Edge` 对象。这些边会指向变量的 `grad_fn`，同时追踪 `output_nr` 来创建遍历图时用到的 id。如果输入变量是叶节点，例如不是由可微函数创建，未设定 `grad_fn` 属性，则会默认设定一个被称为梯度累加器的特殊函数，如上面的代码片段所示。在创建边后，当前创建的 `grad_fn` 节点对象会使用 `set_next_edges` 函数保存它们。这就是所谓的将 `grad_fn` 连接起来，产生计算图。

```
void set_next_edges(edge_list&& next_edges) {  
    next_edges_ = std::move(next_edges);  
    for(const auto& next_edge : next_edges_) {  
        update_topological_nr(next_edge);  
    }  
}
```

现在会执行函数的前向传播，在 `set_history` 执行后会将输出向量连接到 `grad_fn` 节点。

```
inline void set_history(at::Tensor& variable,
                       const std::shared_ptr<Node>& grad_fn) {
    AT_ASSERT(grad_fn);
    if (variable.defined()) {
        // If the codegen triggers this,
        // you most likely want to add your newly added function
        // to the DONT_REQUIRE_DERIVATIVE list in
        // tools/autograd/gen_variable_type.py
        TORCH_INTERNAL_ASSERT(isDifferentiableType(variable.scalar_type()));
        auto output_nr = grad_fn->add_input_metadata(variable);
        impl::set_gradient_edge(variable, {grad_fn, output_nr});
    }
    else {
        grad_fn->add_input_metadata(Node::undefined_input());
    }
}
```

`set_history` 会调用 `set_gradient_edge`，它将 `grad_fn` 和 `output_nr` 复制到这个张量的 `AutogradMeta` 对象。

```
void set_gradient_edge(const Variable& self, Edge edge) {
    auto* meta = materialize_autograd_meta(self);
    meta->grad_fn_ = std::move(edge.function);
    meta->output_nr_ = edge.input_nr;
    // For views, make sure this new grad_fn_ is not
    // overwritten unless it is necessary
    // in the VariableHooks::grad_fn below.
    // This logic is only relevant for custom
    // autograd Functions for which multiple
    // operations can happen on a given Tensor
    // before its gradient edge is set when
    // exiting the custom Function.
    auto diff_view_meta = get_view_autograd_meta(self);
    if (diff_view_meta && diff_view_meta->has_bw_view()) {
        diff_view_meta->set_attr_version(self._version());
    }
}
```

这个张量现在会是另一个函数的输入，上面的步骤会重复，直到完成计算图的构建。

2.3 在图中注册 Python 函数

前面展示了 Autograd 为 ATen 中的函数创建图的过程，但当使用 Python 定制可微函数时，它们也被包含在图中。一个 Autograd 的 Python 函数定义如下所示：


```

class Exp(torch.autograd.Function):

    @staticmethod
    def forward(ctx, i):
        result = i.exp()
        ctx.save_for_backward(result)
        return result

    @staticmethod
    def backward(ctx, grad_output):
        result, = ctx.saved_tensors
        return grad_output * result

# Call the function
Exp.apply(torch.tensor(0.5, requires_grad=True))
# Outputs: tensor(1.6487, grad_fn=<ExpBackward>)

```

上面的代码片段中 Autograd 能在创建图时检测到 Python 函数，这需要归功于 `Function` 类。下面看一下调用 `apply` 时会发生什么。`apply` 定义在 `torch._C._FunctionBase` 类，但这个类并不在 Python 源码中。在 C++ 中定义了 `_FunctionBase`，通过使用 Python C API 来勾住 C 函数到单个 Python 类中。现在寻找一个名为 `THPFunction_apply` 的函数：

```

PyObject *THPFunction_apply(PyObject *cls, PyObject *inputs) {
    // Generates the graph node
    THPObjectPtr backward_cls(PyObject_GetAttrString(cls, "_backward_cls"));
    if (!backward_cls) return nullptr;
    THPObjectPtr ctx_obj(PyObject_CallFunctionObjArgs(backward_cls, nullptr));
    if (!ctx_obj) return nullptr;
    THPFunction* ctx = (THPFunction*)ctx_obj.get();

    auto cdata = std::shared_ptr<PyNode>(new PyNode(std::move(ctx_obj)),
                                             deleteNode);

    ctx->cdata = cdata;

    // Prepare inputs and allocate context (grad fn)
    // Unpack inputs will collect the edges
    auto info_pair = unpack_input<false>(inputs);
    UnpackedInput& unpacked_input = info_pair.first;
    InputFlags& input_info = info_pair.second;

    // Initialize backward function (and ctx)
    bool is_executable = input_info.is_executable;
    cdata->set_next_edges(std::move(input_info.next_edges));
    ctx->needs_input_grad = input_info.needs_input_grad.release();
}

```

```

    ctx->is_variable_input = std::move(input_info.is_variable_input);

    // Prepend ctx to input_tuple, in preparation for static method call
    auto num_args = PyTuple_GET_SIZE(inputs);
    THPObjectPtr ctx_input_tuple(PyTuple_New(num_args + 1));
    if (!ctx_input_tuple) return nullptr;
    Py_INCREF(ctx);
    PyTuple_SET_ITEM(ctx_input_tuple.get(), 0, (PyObject*)ctx);
    for (int i = 0; i < num_args; ++i) {
        PyObject *arg = PyTuple_GET_ITEM(
            unpacked_input.input_tuple.get(), i);
        Py_INCREF(arg);
        PyTuple_SET_ITEM(ctx_input_tuple.get(), i + 1, arg);
    }

    // Call forward
    THPObjectPtr tensor_outputs; {
        AutoGradMode grad_mode(false);
        THPObjectPtr forward_fn(PyObject_GetAttrString(cls, "forward"));
        if (!forward_fn) return nullptr;
        tensor_outputs = PyObject_CallObject(forward_fn, ctx_input_tuple);
        if (!tensor_outputs) return nullptr;
    }

    // Here is where the outputs gets the tensors tracked
    return process_outputs(cls, cdata, ctx, unpacked_input, inputs,
                          std::move(tensor_outputs),
                          is_executable, node);

    END_HANDLE_TH_ERRORS
}

```

这些代码本质上与 ATen 中自动生成的前向函数执行相同的操作：创建 `grad_fn` 对象，收集边将当前 `grad_fn` 与输入张量连接起来，执行 `forward` 函数，将创建的 `grad_fn` 到分派到输出张量的元数据。`grad_fn` 对象通过下面代码创建：

```

// Generates the graph node
THPObjectPtr backward_cls(PyObject_GetAttrString(cls, "_backward_cls"));
if (!backward_cls) return nullptr;
THPObjectPtr ctx_obj(PyObject_CallFunctionObjArgs(backward_cls, nullptr));
if (!ctx_obj) return nullptr;
THPFunction* ctx = (THPFunction*)ctx_obj.get();

auto cdata = std::shared_ptr<PyNode>(new PyNode(std::move(ctx_obj)),
                                     deleteNode);

ctx->cdata = cdata;

```


基本上它要求 Python API 获取一个指针，指向可以执行用户编写的函数的 Python 对象，然后将其包装到一个 `_PyNode` 对象中，这是一个特定 Node 对象，当前向传播期间执行 `apply` 时，使用给定 Python 函数调用 Python 解释器。注意在代码中，`cdata` 作为图的一部分，是实际的 Node 对象。`ctx` 是传递给 Python `forward / backward` 函数的对象，被用户函数和 PyTorch 用来存储 Autograd 的相关信息。和常规 C++ 函数一样，这里也调用 `collect_next_edges` 来追踪 `grad_fn` 对象，但这是在 `unpack_input` 中完成的：

```
template<bool enforce_variables>
std::pair<UnpackedInput, InputFlags> unpack_input(PyObject *args) {
    ...
    flags.next_edges = (flags.is_executable ?
                        collect_next_edges(unpacked.input_vars) :
                        edge_list());
    return std::make_pair(std::move(unpacked), std::move(flags));
}
```

之后通过 `cdata->set_next_edges(std::move(input_info.next_edges))` 将这些边分派给 `grad_fn`，前向函数也通过 Python 解释器的 C API 调用。一旦输出张量被前向传播返回，就在 `process_outputs` 函数内被处理和转换为变量。

```
PyObject* process_outputs(PyObject *op_obj, const std::shared_ptr<PyNode>& cdata,
                          THPFunction* grad_fn, const UnpackedInput& unpacked,
                          PyObject *inputs, THPObjectPtr&& raw_output,
                          bool is_executable, torch::jit::Node* node) {
    ...
    _wrap_outputs(cdata, grad_fn, unpacked.input_vars,
                  raw_output, outputs, is_executable);
    _trace_post_record(node, op_obj, unpacked.input_vars,
                       outputs, is_inplace, unpack_output);
    if (is_executable) {
        _save_variables(cdata, grad_fn);
    }
    ...

    return outputs.release();
}
```

这里 `_wrap_outputs` 负责将前向输出 `grad_fn` 设为新创建的输出，为此调用另一个定义在不同文件中的 `_wrap_outputs` 函数，因此这个过程略微令人困惑：

```

static void _wrap_outputs(const std::shared_ptr<PyNode>& cdata,
                          THPFunction *self,
                          const variable_list &input_vars,
                          PyObject *raw_output,
                          PyObject *outputs,
                          bool is_executable) {
    auto cdata_if_executable = is_executable ? cdata : nullptr;
    ...

    // Wrap only the tensor outputs.
    // This calls csrc/autograd/custom_function.cpp
    auto wrapped_outputs = _wrap_outputs(input_vars, non_differentiable,
                                          dirty_inputs, raw_output_vars,
                                          cdata_if_executable);

    ...
}

```

被调用的 `_wrap_outputs` 负责在输出张量中设定 Autograd 元数据。

```

std::vector<c10::optional<Variable>> _wrap_outputs(const variable_list &input_vars,
                                                  const std::unordered_set<at::TensorImpl*> &non_differentiable,
                                                  const std::unordered_set<at::TensorImpl*> &dirty_inputs,
                                                  const at::ArrayRef<c10::optional<Variable>> raw_outputs,
                                                  const std::shared_ptr<Node> &cdata) {

    std::unordered_set<at::TensorImpl*> inputs;
    ...
    // Sets the grad_fn and output_nr of an output Variable.
    auto set_history = [&](Variable& var, uint32_t output_nr, bool is_input,
                          bool is_modified, bool is_differentiable) {
        // Lots of checks
        if (!is_differentiable) {
            ...
        }
        else if (is_input) {
            // An input has been returned, but it wasn't modified.
            // Return it as a view so that we can attach
            // a new grad_fn to the Variable.
            // Run in no_grad mode to mimic the behavior of the forward.
            {
                AutoGradMode grad_mode(false);
                var = var.view_as(var);
            }
            impl::set_gradient_edge(var, {cdata, output_nr});
        }
    };
}

```



```
        else if (cdata) {  
            impl::set_gradient_edge(var, {cdata, output_nr});  
        }  
};
```

而这正是调用 `set_gradient_edge` 的地方，就是用户编写的 Python 函数及其关联的反向传递函数，被包含在计算图中的方式。